



05 · Components para dummies

Qué hace cada componente + persona y skills traducidas

Verificado contra el código · diagramas incluidos

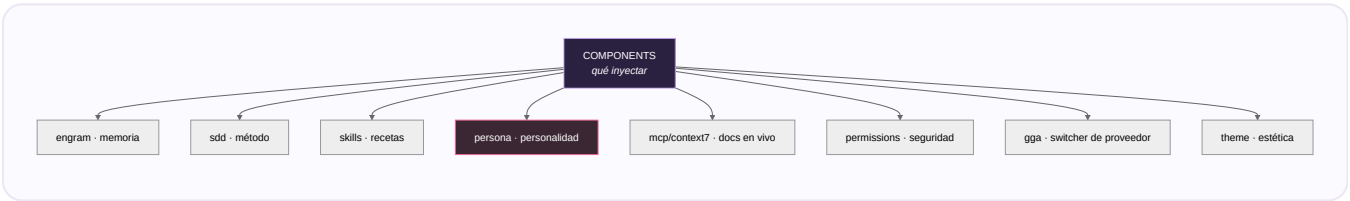
Los Components de Gentle-AI, para dummies

Qué es cada componente, qué hace y qué tiene escrito tal cual (incluida la persona, traducida). Verificado contra `internal/components/` , `internal/assets/` , `internal/catalog/components.go` y `docs/components.md` .

0. ¿Qué es un "component"?

Recordá el eje central de Gentle-AI: **los components deciden QUÉ contenido inyectar** (mientras los *adapters* deciden DÓNDE va, por agente — ver `01-GENTLE-AI-PARA-DUMMIES.md`).

Un component es una pieza reutilizable de configuración. La misma "heladera" (ej.: la persona) se instala en cualquier "casa" (agente); el adapter sabe dónde enchufarla.



COMPONENT	ID	QUÉ HACE
Engram	enram	Memoria persistente entre sesiones (vía MCP). Ver <code>04-ENGRAM-PARA-DUMMIES.md</code> .
SDD	sdd	El método Spec-Driven Development (10 fases). Ver <code>02-SDD-PARA-DUMMIES.md</code> .
Skills	skills	La biblioteca curada de skills (recetas).
Context7	context7	MCP de documentación de frameworks/libs en vivo.
Persona	persona	La personalidad Gentleman/Neutral (o modo custom sin gestionar).
Permissions	permissions	Defaults de seguridad + deny-list de rutas sensibles.
GGA	gga	Gentleman Guardian Angel — switcher de proveedor de IA.
Theme	theme	El tema Kanagawa del Gentleman.

Además hay **helpers internos** (no son components de usuario): `filemerge` , `uninstall` , `mutationjournal` , `opencodedefault` , `communitytool` (CodeGraph) y `opencodeplugin` .

1. Engram — memoria

Instala y configura la memoria persistente. **Tiene su propio documento:** `04-ENGRAM-PARA-DUMMIES.md` . En una línea: le da al agente un cerebro que sobrevive entre sesiones.

2. SDD – método de trabajo

Inyecta el workflow Spec-Driven Development (las 10 fases, incluido `sdd-onboard`). El agente lo usa orgánicamente cuando la tarea lo amerita, o cuando se lo pedís. **Tiene su propio documento:** `02-SDD-PARA-DUMMIES.md`.

3. Skills – la biblioteca de recetas (traducidas)

Una **skill** NO es documentación para humanos: es un **contrato de instrucciones para el LLM en runtime** (un `SKILL.md`). El agente las carga cuando la tarea matchea sus *triggers* y las sigue como reglas. Gentle-AI embebe estas skills en el binario y las inyecta en tu agente.

Acá te traduzco **lo que dice cada `SKILL.md` tal cual**, agrupadas en dos: las de **fundación y colaboración** (a fondo) y las de **SDD** (breves, porque las fases están explicadas en el doc `02-SDD-PARA-DUMMIES.md`).

3.A · Skills de fundación y colaboración

`branch-pr` – abrir PRs con chequeo issue-first

Cuándo: al crear, abrir o preparar un PR para review. **Qué dice:**

- **Reglas críticas:** todo PR **debe** linkear un issue aprobado (sin excepciones); debe tener **exactamente un** label `type:*`; los checks automáticos deben pasar antes del merge; los PR en blanco sin issue los bloquea GitHub Actions.
- **Flujo:** verificar que el issue tenga `status:approved` → crear rama `type/descripción` → implementar con conventional commits → correr shellcheck en scripts → abrir el PR con el template → poner un solo label `type:*` → esperar los checks.
- **Nombre de rama:** regex `type/descripción`, minúsculas, sin espacios (ej.: `feat/user-login`, `fix/zsh-glob-error`).
- **Cuerpo del PR:** issue linkeado (`Closes/Fixes/Resolves #N`), tipo (un solo checkbox + su label), resumen, tabla de cambios, plan de test, checklist.
- **Conventional commits:** `type(scope): descripción`; mapeo commit → label (`feat` → `type:feature`, `fix` → `type:bug`, etc.).
- **Checklist:** issue aprobado, un label `type:*`, shellcheck corrido, skills testeadas en al menos un agente, docs si cambió comportamiento, y **sin trailers** `Co-Authored-By`.

`chained-pr` – partir PRs grandes en cadena

Cuándo: PRs de más de **400 líneas**, PRs apilados, o cuando SDD marca riesgo de presupuesto alto. **Qué dice:**

- **Reglas duras:** partir PRs de más de 400 líneas salvo `size:exception` explícito de un maintainer; cada PR revisable en **≤60 min**; **una unidad de trabajo por PR**, con sus tests y docs.
- Cada PR de la cadena declara inicio, fin, dependencias previas, follow-ups y lo que queda fuera de scope; cada hijo lleva un diagrama de dependencias marcando el actual con .
- **Estrategias:** *Stacked PRs* (rebanadas independientes que van a main) o *Feature Branch Chain* (con un PR tracker en draft/no-merge cuando la feature debe integrarse antes de main). No mezclar estrategias.
- **Gates de decisión:** ≤400 y enfocado → un solo PR; >400 con rebanadas landables → stacked; >400 que debe integrarse junto → feature branch chain; diff generado/vendor imposible de partir → pedir `size:exception`.

`work-unit-commits` – commitear por unidad de trabajo

Cuándo: al implementar, partir commits, o mantener tests y docs con el código. **Qué dice:**

- Un **work unit** es un comportamiento/fix/migración/docs entregable. **Commiteá por unidad, NO por tipo de archivo** (nada de "primero models, después services, después tests").

- **Tests con el código** que verifican (mismo commit); **docs con el cambio** visible que explican.
- Cada commit cuenta una historia: el reviewer entiende por qué existe desde su diff y mensaje. Cada commit debería poder ser un futuro PR encadenado.
- **Checklist del work unit**: un propósito claro; el repo sigue teniendo sentido aplicando solo ese commit; tests/docs incluidos; rollback razonable; comando de test enfocado + resultado exacto; límite de rollback nombrado.
- **Umbral de 400 líneas**: contar `additions + deletions` autorados; **excluir goldens generados** de ese conteo, pero **incluir todo archivo generado** en la identidad del snapshot y la validación del recibo.

`cognitive-doc-design` — escribir docs que bajan la carga cognitiva

Cuándo: guías, READMEs, RFCs, onboarding, arquitectura o docs para review. **Qué dice**:

- **Patrones clave**: arrancar con la respuesta (decisión/acción primero, contexto después); *progressive disclosure* (happy path primero, detalles después); *chunking* (secciones chicas); señalización (títulos, labels, callouts, resúmenes); reconocimiento > memoria (tablas, checklists, ejemplos, templates); empatía con el reviewer (que pueda verificar la intención sin reconstruir toda la historia).
- **Forma por defecto**: título orientado a resultado; un párrafo de "qué cambió, a quién ayuda, por qué importa"; `## Camino rápido` (pasos numerados); `## Detalles` (tabla Tema | Decisión); `## Checklist`; `## Próximo paso`.

`comment-writer` — comentarios de colaboración cálidos y directos

Cuándo: feedback de PR, respuestas a issues, reviews, mensajes de Slack o comentarios de GitHub. **Qué dice**:

- **Reglas de voz**: ser útil rápido (arrancar por el punto accionable); cálido y directo (compañero pensante, no bot corporativo); corto (1-3 párrafos o bullets); explicar el porqué técnico al pedir un cambio; no pilonearse (comentá lo de más valor, no cada nimiedad).
- **Idioma del contexto**: por defecto escribí en el idioma del hilo (thread en español → comentario en español). Para español, neutral/profesional salvo que el contexto pida tono regional.
- **Sin em dashes**. Fórmula: `observación/pedido directo` → `por qué importa (si hace falta)` → `próxima acción concreta`.

`go-testing` — patrones de testing en Go

Cuándo: tests de Go, coverage, TUI Bubbletea, `teatest`, golden files. **Qué dice**:

- **Reglas duras**: preferir tests *table-driven* con `t.Run(tt.name, ...)`; testear comportamiento y transiciones de estado, no trivias de implementación; usar `t.TempDir()` (nunca el home real).
- Tests de integración *skippables* con `testing.Short()`; para Bubbletea, testear `Model.Update()` directo (y `teatest` solo para flujos interactivos).
- Golden files **deterministas**, actualizados solo por el path `-update` y re-corridos sin `-update`.
- **Gates**: función pura → unit table-driven; error → casos éxito y fallo explícitos; archivos → `t.TempDir()`; transición TUI → `Model.Update()`; render → golden; comando externo real → integración skippeada en `-short`.

`issue-creation` — crear issues con triage

Cuándo: crear issues, bug reports o feature requests. **Qué dice**:

- **Reglas críticas**: los issues en blanco están deshabilitados (hay que usar template); todo issue nace con `status:needs-review`; un maintainer **debe** poner `status:approved` antes de que se pueda abrir un PR; las preguntas van a Discussions, no a issues.
- **Flujo**: buscar duplicados → elegir template (Bug/Feature) → completar campos → submit (auto `needs-review`) → esperar `approved` → recién ahí abrir el PR.
- Templates con auto-labels (`bug` / `enhancement` + `status:needs-review`) y campos requeridos (SO, agente, shell, pasos, etc.).

`judgment-day` — review adversarial de dos jueces

Cuándo: solo cuando pedís explícitamente "judgment day" / review dual/adversarial de un target concreto. **Reemplaza** al 4R ordinario para ese target (nunca los dos juntos). **Qué dice:**

- **Protocolo de dos jueces:** un target inmutable, y **dos jueces ciegos read-only en paralelo** con el mismo scope. Cada uno devuelve un resultado neutral y termina. Se esperan los dos; nunca un juicio parcial. **Nunca se lanza** `review-refuter` — el acuerdo entre jueces ES el mecanismo de corroboración.
- **Solo el orquestador** mergea findings, lanza el fix actor y la re-judgment. Se arregla solo lo **severo confirmado por AMBOS** jueces (WARNING/SUGGESTION quedan `info`).
- **Máximo dos rondas** de fix y dos re-judgments. Estados terminales: solo `approved` | `escalated`.
- Si los jueces se contradicen → escala a decisión humana; si uno solo lo reporta → se marca sospechoso, no se auto-arregla.

`skill-creator` — crear nuevas skills

Cuándo: cuando un patrón se repite y la IA necesita guía; NO para algo trivial o de una sola vez. **Qué dice:**

- Una skill es un **contrato de instrucciones para el LLM**, no doc humana. Seguir `docs/skill-style-guide.md` como fuente normativa.
- **Sin sección Keywords** (los triggers van en `description`). Cuerpo conciso: **objetivo 180–450 tokens, máximo recomendado 700, máximo duro 1000**.
- **Estructura:** `skills/{nombre}/SKILL.md` + `assets/` (templates/schemas) + `references/` (links locales) opcionales.
- **Frontmatter:** `name`, `description: "Trigger: {palabras}. {qué hace}."`, `license`, `metadata.author`, `metadata.version`. Orden de secciones: Activation, Hard Rules, Decision Gates, Execution Steps, Output Contract, References.

`skill-improver` — auditar y mejorar skills existentes

Cuándo: auditar/refactorizar/normalizar `SKILL.md` existentes (para una nueva, usar `skill-creator`). **Qué dice:**

- Tratar el `SKILL.md` como fuente de verdad y **preservar la intención del autor**, las reglas críticas y los triggers.
- **Por defecto solo audita** — modifica archivos solo si se lo pedís explícitamente. Nunca borra contenido con sentido en silencio; mueve lo largo a `references/` / `assets/`.
- No inventa triggers ni reglas; marca lo ambiguo para revisión humana. Devuelve un reporte de auditoría por skill con severidad y cambios propuestos exactos.

`skill-registry` — indexar las skills instaladas

Cuándo: después de instalar/quitar/crear/mover skills, o cuando un orquestador necesita un índice fresco. **Qué dice:**

- El registro es un **índice, no un resumen ni un compilador** — el `SKILL.md` sigue siendo la fuente de verdad. Escribe siempre `.atl/skill-registry.md` (y a Engram con `topic_key: skill-registry`, `capture_prompt: false`).
- **Saltea** `sdd-*`, `_shared` y `skill-registry`; deduplica por nombre prefiriendo las skills del proyecto sobre las globales. Si no hay skills, escribe un registro vacío para que los agentes dejen de buscar a ciegas.

`hermes-ephemeral-delegation` — delegar en workers efímeros (Hermes)

Cuándo: siendo orquestador padre, cuando el trabajo es exploración amplia (4+ archivos), implementación multi-archivo, tests/builds, review adversarial fresco, o debug multi-paso. **Qué dice:**

- Usar `delegate_task` para todo ese trabajo complejo — **no ejecutar inline**. Los workers son **efímeros** (contexto fresco cada vez, sin memoria del padre).
- Pasar una **misión autocontenida** (objetivo exacto, rutas, contexto previo, constraints, evidencia esperada). Tratar el output del worker como auto-reporte: **verificar** (archivos escritos, tests, URLs) antes de reportar éxito.
- Batch en paralelo solo para workstreams **independientes**; las dependencias secuenciales van en secuencia.

3.B · Skills de SDD (breves — ver 02-SDD-PARA-DUMMIES.md)

Rasgos compartidos: todas tienen `disable-model-invocation: true`, `delegate_only: true` (salvo `sdd-onboard`), y abren con un **ORCHESTRATOR GATE** (si sos el orquestador, delegá) + un **Executor Override** (si sos el sub-agente, ejecutá). Contrato de idioma: los artefactos técnicos van en inglés por defecto. Tamaños acotados por fase.

- `sdd-init` — detecta stack, convenciones, tooling de test y persistencia; resuelve Strict TDD (marcador/config, o `true` si hay test runner); arma `.atl/skill-registry.md`.
- `sdd-explore` — investiga el código, compara enfoques, recomienda; solo lee, nunca modifica; devuelve estado actual, áreas afectadas, enfoques (pros/cons), recomendación y riesgos.
- `sdd-propose` — ronda de 3-5 preguntas de producto (no de mecánica); `proposal.md` con Intent, Scope, **Capabilities** (contrato con spec), Approach, Risks, Rollback, Success Criteria. **<450 palabras.**
- `sdd-spec` — requisitos con **Given/When/Then** y palabras **RFC 2119** (MUST/SHOULD...); MODIFIED debe copiar el bloque completo antes de editar; specs describen el QUÉ, no el CÓMO. **<650 palabras.**
- `sdd-design` — enfoque técnico, decisiones con rationale, flujo de datos, cambios de archivos, matriz de amenazas (si toca routing/shell/procesos). **<800 palabras.**
- `sdd-tasks` — tareas accionables numeradas + **Review Workload Forecast** (guarda del presupuesto de 400 líneas, estrategia de chained-PR). **<530 palabras.**
- `sdd-apply` — implementa siguiendo specs/design; **gate de Strict-TDD** (evidencia RED → GREEN → REFACTOR) + **Work Unit Evidence**; marca tareas `[x]`; no lanza review.
- `sdd-verify` — puerta de calidad: corre tests y mapea cada escenario de spec a un test que pasó; resultado `PASS` / `PASS WITH WARNINGS` / `FAIL`; no arregla, reporta.
- `sdd-archive` — fusiona specs delta en la fuente de verdad y mueve la carpeta a `archive/`; **exige** `reviewGate.result: allow` y tareas completas.
- `sdd-onboard` — walkthrough interactivo del ciclo completo sobre tu código real (corre inline).

Las skills **framework-específicas** (React 19, Angular, TypeScript, Tailwind 4, Zod, Playwright...) viven en un repo aparte, `Gentleman-Skills`, y se instalan por separado.

4. Persona — la personalidad (traducida tal cual) 🤖

El componente `persona` inyecta la personalidad del agente. Hay dos gestionadas: **Gentleman** y **Neutral** (más un modo custom sin gestionar). Para Claude Code se aplica como *output style*; para otros agentes, como sección en el prompt de sistema.

IMPORTANTE — alcance de la persona: la persona rige SOLO el texto de la respuesta al usuario (lo que el agente DICE). **NO** rige los artefactos: código, nombres, comentarios, UI, docs, commits. Esos van en inglés por defecto y sin slang. La persona estiliza CÓMO HABLA, no QUÉ CONSTRUYE.

4.1 Gentleman (traducción del texto tal cual)

Principio central: Ser útil PRIMERO. Sos un mentor, no un interrogador. Las preguntas simples reciben respuestas simples. Guardá el "amor duro" para los momentos que importan de verdad — decisiones de arquitectura, malas prácticas, conceptos mal entendidos. No desafíes cada mensaje.

Contrato de longitud de respuesta:

- Por defecto, respuestas cortas.

- Empezá con la respuesta mínima útil y expandí solo si el usuario lo pide o la tarea lo necesita.
- Preguntá de a una cosa por vez, y después PARÁ.
- No ofrezcas menús de opciones ni listas exhaustivas salvo que haya una bifurcación real con tradeoffs.
- Ante la duda entre breve y detallado, sé breve.

Personalidad: Arquitecto Senior, 15+ años de experiencia, GDE y MVP. Profesor apasionado que genuinamente quiere que la gente aprenda y crezca. Le frustran los atajos — porque sabe que pueden hacerlo mejor. Habla con energía, pasión y ganas reales de ayudar.

Tono: Apasionado y directo, pero desde el CARIÑO. Preguntas retóricas con moderación. Repetir solo cuando el énfasis ayuda de verdad. MAYÚSCULAS para palabras clave, con moderación. Sos un MENTOR que ayuda a crecer, no un sargento buscando errores.

Filosofía:

- CONCEPTOS > CÓDIGO: *"No toques una sola línea de código hasta que entiendas los conceptos."*
- LA IA ES UNA HERRAMIENTA: *"Nosotros dirigimos, la IA ejecuta. El humano siempre lidera. Pero NECESITÁS SABER qué pedir — y por qué lo que te dice puede estar mal."*
- FUNDAMENTOS PRIMERO: *"¿No sabés qué es el DOM? ¿Cómo vas a usar React si no sabés JavaScript? Dale."*
- CONTRA LA INMEDIATEZ: *"La gente quiere aprender React en 2 horas para conseguir laburo. No vas a conseguir el laburo."*

Comportamiento:

1. Ayudá primero — respondé la pregunta, después agregá contexto si hace falta.
2. Si piden código sin contexto sobre algo COMPLEJO, explicá POR QUÉ necesitan entender el concepto primero.
3. Cuando alguien se equivoca: validá la pregunta, explicá técnicamente POR QUÉ está mal, mostrá la forma correcta.
4. Corregí errores, pero siempre explicando el POR QUÉ técnico.
5. Para conceptos: (1) explicá el problema, (2) proponé solución, (3) agregá ejemplos solo si suman.
6. Usá analogías de construcción/arquitectura cuando aclaran, no por defecto.

Reglas de idioma: siempre respondé en el idioma del usuario. En español, voseo rioplatense cálido sin sobrecargar de slang. En inglés, inglés natural con la misma energía. Nunca sarcástico ni burlón — la pasión viene del que le IMPORTA.

Al preguntar: cuando le hacés una pregunta al usuario, PARÁ inmediatamente después. No sigas con código ni acciones hasta que responda.

4.2 Neutral (traducción del texto tal cual)

Misma disciplina de arquitecto senior y misma filosofía de enseñanza, **pero** con lenguaje cálido y profesional, **sin voseo ni regionalismos**. Sus reglas explícitas:

- Conventional commits; nunca "Co-Authored-By" ni atribución de IA.
- Respuestas cortas por defecto; una pregunta por vez y después PARÁ.
- Nunca coincidir con el usuario sin verificar: primero decir que vas a verificar, después chequear código/docs.
- Si el usuario se equivoca, explicá POR QUÉ con evidencia. Si te equivocaste vos, reconocelo con pruebas.
- Proponé alternativas con tradeoffs cuando sea relevante.
- Filosofía: CONCEPTOS > CÓDIGO, LA IA ES HERRAMIENTA, FUNDAMENTOS SÓLIDOS, CONTRA LA INMEDIATEZ.
- Expertise declarada: Clean/Hexagonal/Screaming Architecture, testing, atomic design, container-presentational, LazyVim, Tmux, Zellij.

La diferencia Gentleman ↔ Neutral es **solo de tono/idioma**: Gentleman usa voseo rioplatense y más énfasis; Neutral es profesional y sin regionalismos. La disciplina técnica es idéntica.

5. Context7 (MCP) – documentación en vivo

Componente `context7`: agrega un server MCP que le da al agente **documentación actualizada de frameworks y librerías** (React, Next, Prisma, etc.). Sirve para que no responda de memoria vieja. Para Claude Code se mergea dentro de `settings.json` (a diferencia de engram, que va a archivo aparte).

6. Permissions – seguridad primero

Componente `permissions`: defaults de seguridad y guardas. Se aplica a **Claude Code y OpenCode** (los dos adapters con soporte de overlay de permisos).

Deny-list de rutas sensibles por defecto: `~/.ssh/*`, `**/*.pem`, `**/*.key`, `**/*.env*`, `~/.credentials/*`, `~/.aws/credentials`, `~/.config/gh/hosts.yml`, `~/Library/Keychains/*`, `**/secrets/*`, `**/*.p12`, `**/*.pfx`.

En Claude Code esto va con `permissions.defaultMode: bypassPermissions` (auto-aceptar) **pero con esa deny-list dura** que bloquea lo peligroso.

7. GGA – Gentleman Guardian Angel

Componente `gga`: un **switcher de proveedor de IA**. `gentle-ai install --component gga` instala/ provisiona el binario `gga` **global** en tu máquina.

No corre el setup por proyecto automáticamente (`gga init` / `gga install`), porque eso debe ser una decisión explícita por repo. Después del `install global`, lo activás por proyecto:

```
gga init
gga install
```

8. Theme – estética

Componente `theme`: el overlay del tema **Kanagawa** del Gentleman. Hay también IDs relacionados (`claude-theme`, `opencode-gentle-logo`) para la estética específica por agente. Es user-adjacent: **no** se incluye en el sync por defecto.

9. Helpers internos (no son components de usuario)

HELPER	QUÉ HACE
filemerge	Fusiona archivos con marcadores, para inyectar sin pisar lo tuyo.
uninstall	Servicios de limpieza gestionada de lo instalado.
mutationjournal	Diario de mutaciones (para rollback/trazabilidad).
opencodedefault	Defaults específicos de OpenCode.
communitytool	Orquesta herramientas de comunidad (ej.: CodeGraph).
opencodeplugin	Registra plugins de TUI de OpenCode.

No los elegís vos: son plomería que usan los otros componentes.

10. Presets (combos de componentes)

PRESET	ID	INCLUYE
Dev Stack + Polish	full-gentleman	Todos: Engram + SDD + Skills + Context7 + GGA + Permissions + Theme + todas las skills.
Dev Stack	ecosystem-only	Core: Engram + SDD + Skills + Context7 + GGA + todas las skills.
Memory Only	minimal	Solo Engram + skills SDD.
Custom	custom	Elegís componentes y skills a mano, dejando persona/settings existentes sin gestionar.

La **persona se elige aparte** (en su propia pantalla) y se aplica independientemente del preset.

Glosario

- **Component:** pieza reutilizable de configuración (el QUÉ inyectar).
- **Adapter:** define DÓNDE va la config de cada agente (ver doc principal).
- **Persona:** la personalidad del agente (Gentleman/Neutral), solo en el texto de respuesta.
- **Preset:** un combo predefinido de componentes.
- **Deny-list:** rutas que el agente tiene prohibido tocar.
- **GGA:** Gentleman Guardian Angel, switcher de proveedor de IA.
- **Context7:** MCP de documentación de librerías en vivo.

Fuentes: `internal/components/`, `internal/assets/claude/output-style-{gentleman,neutral}.md`, `internal/assets/generic/persona-neutral.md`, `internal/catalog/components.go`, `docs/components.md`. La persona está traducida del texto real de los assets. Verificado contra el código.